

Brain Train — Project Summary & Design Decisions

Project Overview

Brain Train is a premium AI-powered Olympiad preparation app for an advanced Grade 2 student in India.

The objective is to create an experience that feels like a **personal AI tutor** rather than a traditional quiz application.

The app will be developed using **Claude Code** while using **Google Gemini** as the exclusive runtime AI provider.

Core Product Vision

The app should:

- Feel magical, playful and premium.
- Encourage curiosity instead of exam anxiety.
- Function almost entirely offline.
- Become smarter over time by remembering the student's progress.
- Make Olympiad preparation feel like an exciting journey.

Tagline:

All Aboard for Olympiad Success!

Branding

App Name

Brain Train

The entire application revolves around a train journey metaphor.

Branding Elements

App Feature	Branding
Welcome	 All Aboard!
Start Quiz	Depart Now
Subject Selection	Choose Your Destination
Hall of Fame	Engineer's Log
Mistake Bucket	Repair Yard
Perfect Score	Express Service!
Quiz Complete	You reached the station!

Technology Stack

- React Native (Expo)
- TypeScript
- NativeWind
- Zustand
- Android-first
- Portrait-only
- Storage abstraction
- AsyncStorage/MMKV on Android
- Optional localStorage implementation for Web

Runtime AI Strategy

The development agent is:

- Claude Code

The runtime AI provider is:

- Google Gemini

The application must never substitute another provider.

Gemini Configuration

The user provides:

- Gemini API Key
- Gemini Model

Default:

```
gemini-3.5-flash
```

The model must be configurable from Settings so future Gemini upgrades require no code changes.

Offline-First Architecture

One of the biggest architectural improvements discussed was making the application genuinely offline after quiz generation.

Offline Guarantee

Once the quiz package has been downloaded, the device may be switched into Airplane Mode and the following must continue working:

- Answer validation
- Scoring
- Lifelines
- Review mode
- Explanations
- Hall of Fame
- Daily Streak
- Mistake Bucket
- WhatsApp sharing

No further Gemini calls are permitted.

Gemini API Calls

Maximum:

Call 1 (Optional)

Session initialization.

May generate:

- personalized greeting
 - streak acknowledgement
 - leaderboard commentary
-

Call 2 (Mandatory)

Generate the complete Quiz Session Package.

After this:

No more API calls.

Everything becomes local.

Quiz Session Package

Instead of only returning questions, Gemini returns a complete package.

It includes:

- metadata
- introduction
- coach tip
- score feedback
- achievement messages
- questions
- explanations
- backup question
- answer key

This enables complete offline gameplay.

JSON Improvements

Instead of only:

```
questions[]
```

The JSON now includes:

- metadata
- intro
- coach_tip
- score_feedback
- achievement_messages
- questions
- answer_key
- backup_question

Every question includes:

- id
- question
- options
- correct_index
- explanation
- concept

A separate

```
answer_key[]
```

is also returned.

The app validates:

```
question.correct_index == answer_key[index]
```

before accepting the payload.

Quiz Generation

Gemini receives:

- Student name
- Subject
- Difficulty
- Current streak

- Hall of Fame summary
- Highest score
- Mistake Bucket summary
- Weak topics

Instead of generating random questions, Gemini generates a personalized coaching session.

Gameplay

- One question per screen
- Offline scoring
- Large touch cards
- Portrait only

Lifelines:

- 50-50
- Flip Question

Each usable once.

Progress Tracking

The application stores locally:

- Daily streak
- Hall of Fame
- Settings
- Mistake Bucket
- Quiz history

Mastery Mode removes questions after they are answered correctly.

Duplicates are not stored.

UX Philosophy

One of the most important additions was elevating UX to a first-class requirement.

Rather than instructing Claude Code to "make a nice UI", the specification now explicitly requires a premium, child-first experience.

Design principles include:

- Soft pastel palette
- Large touch targets
- Rounded cards
- Smooth spring animations
- Premium micro-interactions
- Delightful illustrations
- Friendly typography
- Zero clutter
- Mobile-first portrait design

The experience should feel comparable in polish to leading educational apps while maintaining Brain Train's unique identity.

Branding Experience

The app should feel like boarding a magical train.

Examples:

- "All Aboard!"
- "Depart Now"
- "Choose Your Destination"
- "Express Service!"
- "Repair Yard"

The branding reinforces learning as a journey rather than a test.

Deliverables

Claude Code should produce:

- Clean Git repository
- Modular architecture
- Tests
- README
- Build scripts
- APK-ready project

The workflow pauses:

- before GitHub push
- before APK generation

to await user approval.

Final PRD

The final Product Requirements Document includes:

- Product vision
 - Offline-first architecture
 - Runtime AI strategy
 - Branding guide
 - Brain Train UX principles
 - Complete Gemini contract
 - JSON validation
 - Acceptance tests
 - Project structure
 - Build workflow
 - APK preparation
 - Future-proof Gemini configuration
-

Final HTML PRD

The latest generated HTML specification is:

Brain_Train_Master_PRD_v3.html

It contains:

- Brain Train branding
- Premium UX guidance
- Offline-first architecture
- Two-call Gemini strategy
- Complete Quiz Session Package
- JSON contract
- Validation rules
- Acceptance tests
- Deliverables
- APK preparation requirements
- Visual Design requirements emphasizing a delightful, production-quality child-first experience.

This HTML document represents the current master specification for the project and should be used as the primary reference when implementing **Brain Train** with Claude Code.